



Webhooks — Complete Notes

Event-driven HTTP callbacks · "Don't call us, we'll call you"

Topic Webhooks

Model Push (reverse API)

Transport HTTP POST

Use Integrations

A reader-friendly, all-in-one reference for webhooks.

Table of Contents

1. What is a Webhook
2. Webhooks vs Polling
3. How Webhooks Work
4. Anatomy of a Webhook Request
5. Security
6. Reliability (Retries & Idempotency)
7. Receiving a Webhook
8. Use Cases & Providers
9. Webhooks vs WebSockets vs SSE vs Polling
10. Testing & Debugging
11. Best Practices
12. Quick Mental Model
13. Common Interview Questions

1. What is a Webhook

A **webhook** is a way for one system to **notify another in real time when an event happens**, by sending an **HTTP POST** to a URL you registered. Instead of you repeatedly asking "anything new?", the provider **pushes** the event to your endpoint the moment it occurs.

 **Mental model:** a webhook is a "**reverse API**" — with a normal API *you* call the provider; with a webhook the *provider* calls *you*. It's the "**don't call us, we'll call you**" pattern: register a URL, get POSTed when something happens.

Common uses: "payment succeeded" (Stripe), "push to repo" (GitHub → CI), "message posted" (Slack), "form submitted", "build finished" — anything event-driven across services.

2. ⚖️ Webhooks vs Polling

The problem webhooks solve — avoiding **polling**:

	Polling	Webhook
Who initiates	You repeatedly ask the API	Provider pushes to you
Latency	Delayed (until next poll)	Near real-time
Efficiency	Wasteful (mostly empty responses)	Efficient (only on real events)
Direction	Client → server (pull)	Server → client (push)
Cost	Many wasted requests	Minimal

Polling = checking your mailbox every 5 minutes. Webhook = the postman rings your bell when mail arrives. Webhooks win for real-time, efficiency, and scale — at the cost of needing a **public endpoint** to receive them.

3. 🔄 How Webhooks Work

1. REGISTER
|
▼
You give the provider a URL + the events you want
(e.g. `https://api.you.com/hooks/stripe`)
2. EVENT HAPPENS
|
▼
Something occurs at the provider (payment succeeds)
3. PROVIDER POSTS
|
▼
Provider sends HTTP POST {event payload} → your URL
4. YOU RESPOND
|
▼
Your endpoint validates, returns 2xx fast, then processes (often async). Non-2xx → provider retries.

1. **Register** the endpoint URL + which events to subscribe to (via the provider's dashboard/API).
2. When the **event occurs**, the provider builds a payload.
3. It sends an **HTTP POST** to your URL with the event data.
4. Your server **verifies + acknowledges with a 2xx** quickly; failures trigger **retries**.

4. 📦 Anatomy of a Webhook Request

A webhook is just an HTTP POST:

```
POST /hooks/github HTTP/1.1
Host: api.you.com
Content-Type: application/json
X-GitHub-Event: push
X-Hub-Signature-256: sha256=4f3a...      ← signature for verification
X-GitHub-Delivery: 12345-abcde          ← unique delivery id (idempotency)

{
  "ref": "refs/heads/main",
  "repository": { "name": "cloud-notes" },
  "pusher": { "name": "alice" }
}
```

- **Method:** almost always **POST**. **Body:** JSON (sometimes form-encoded).
- **Headers** carry the **event type**, a **signature**, and a **unique delivery/event ID**.
- Your endpoint must be **publicly reachable over HTTPS**.

5. Security

Because anyone could POST to your public URL, you must **verify** webhooks are genuine:

- **Signature verification (HMAC)** — the provider signs the payload with a **shared secret**; you recompute the HMAC over the **raw body** and compare. Mismatch → reject. (*Most important control.*)
- **HTTPS only** — so payloads (and signatures) aren't intercepted/tampered.
- **Verify the timestamp** to reject **replayed** old requests (replay protection).
- **Use a hard-to-guess URL** + optional shared token, but never rely on secrecy of the URL alone.
- **IP allow-listing** if the provider publishes fixed source IPs.
- **Validate the payload** (don't trust its contents blindly); use **constant-time** comparison for signatures.

```
import hmac, hashlib

def verify(secret, raw_body, signature):
    digest = hmac.new(secret.encode(), raw_body, hashlib.sha256).hexdigest()
    return hmac.compare_digest(f"sha256={digest}", signature)  # constant-time
```

6. Reliability (Retries & Idempotency)

Networks fail, so webhook delivery is "**at least once**" — design for it:

- **Respond fast with 2xx** — acknowledge **before** heavy processing (do the work async via a queue). Slow endpoints cause timeouts → retries.
- **Retries** — providers retry on non-2xx/timeout with **exponential backoff**; persistent failures may be disabled or sent to a dead-letter view.
- **Idempotency** — the **same event can arrive more than once** (or out of order). Store the **delivery/event ID** and skip duplicates so reprocessing is safe.
- **Ordering is not guaranteed** — don't assume event B arrives after event A; use timestamps/sequence in the payload if order matters.

- Persist incoming events first (e.g. to a queue/DB), then process — so a crash mid-processing doesn't lose the event.

7. 🛠️ Receiving a Webhook

A minimal receiver (Express / Node):

```
app.post('/hooks/stripe',
  express.raw({ type: 'application/json' })), // need the RAW body for HMAC
(req, res) => {
  const sig = req.headers['x-signature'];
  if (!verify(SECRET, req.body, sig)) return res.sendStatus(401); // reject fakes

  const event = JSON.parse(req.body);
  enqueue(event);           // hand off to async processing
  res.sendStatus(200);       // ack FAST (don't block on processing)
});
```

- Capture the **raw body** for signature checks (parsing first can break the HMAC).
- **Return 2xx immediately**, then process asynchronously (queue/worker).
- Be defensive: handle unknown event types, malformed payloads, and duplicates.

8. 🌐 Use Cases & Providers

Provider	Example event
GitHub / GitLab	push, pull request, issue → trigger CI/CD
Stripe / PayPal	<code>payment_succeeded</code> , <code>invoice.paid</code>
Slack / Discord	incoming messages, slash commands
Shopify	order created, inventory changed
Twilio / SendGrid	SMS received, email delivered/bounced
CI tools	build finished → notify Slack

Webhooks are the glue of **integrations** and **event-driven architecture** — they connect SaaS tools without polling. (On AWS, **SNS** can deliver to HTTP/S endpoints, and **API Gateway** → **Lambda** is a common way to *receive* webhooks.)

9. Webhooks vs WebSockets vs SSE vs Polling

	Direction	Connection	Best for
Polling	Client pulls	Repeated requests	Simple, infrequent checks
Webhook	Server → server push	One-off HTTP POST per event	Server-to-server event notifications
WebSocket	Bi-directional	Persistent, full-duplex	Live chat, games, collaborative apps
SSE (Server-Sent Events)	Server → client stream	Persistent one-way	Live feeds/notifications to a browser

Webhook = server-to-server, fire-and-forget POST per event. WebSocket/SSE keep a **live connection to a client** (usually a browser); webhooks don't hold a connection.

10. Testing & Debugging

- **Inspect deliveries:** tools like **webhook.site**, **RequestBin**, or the provider's "recent deliveries" log show the exact payload/headers and let you **redeliver**.
- **Local development:** expose your localhost with a tunnel (**ngrok**, **cloudflared**) so the provider can reach `http://localhost`.
- **Replay** failed deliveries from the provider dashboard while fixing your handler.
- Log the **delivery ID + status** for every received hook to trace issues.

11. Best Practices

- **Verify every webhook** (HMAC signature over the raw body, constant-time compare); **HTTPS only**.
- **Acknowledge with 2xx fast**, then process **asynchronously** (queue + worker).
- **Be idempotent** — dedupe on the delivery/event ID; assume at-least-once delivery.
- **Don't assume ordering**; use timestamps/sequence if it matters.
- **Persist first, process later** so a crash doesn't lose events.
- **Handle retries** gracefully; return clear 4xx for permanent failures, 5xx for transient.
- **Protect against replay** (timestamp check) and validate/limit payload size.
- **Monitor & log** deliveries; provide a way to **replay** missed events.

12. Quick Mental Model

- **Webhook = reverse API:** register a URL, and the provider **POSTs you an event** when something happens — push, not poll.
- Beats **polling** for real-time + efficiency, but needs a **public HTTPS endpoint**.
- Flow: **register** → **event** → **provider POSTs** → **you verify + 2xx fast** → **process async**.
- **Verify with HMAC signature** over the raw body (the key security control); HTTPS + replay protection.
- Delivery is **at-least-once** → be **idempotent** (dedupe on event ID); don't assume **ordering**.

- **Ack quickly, process asynchronously** (queue) so slow work doesn't trigger retries.
 - vs **WebSocket/SSE** (persistent client connections) — webhooks are one-off **server-to-server** POSTs.
-

13. ? Common Interview Questions

Rapid-fire questions interviewers ask about webhooks:

- **Q: What is a webhook?** — A user-defined HTTP callback: a provider POSTs event data to your registered URL when an event occurs (a "reverse API").
 - **Q: Webhook vs API polling?** — Polling repeatedly asks the API for changes (wasteful, delayed); a webhook is pushed by the provider in near real-time only when an event happens.
 - **Q: How do you secure a webhook endpoint?** — Verify the HMAC signature over the raw body with a shared secret (constant-time compare), require HTTPS, check the timestamp for replay protection, and validate the payload.
 - **Q: Why verify the signature on the raw body?** — Parsing/re-serializing changes bytes and breaks the HMAC; you must hash the exact raw payload the provider signed.
 - **Q: How do you handle duplicate webhook deliveries?** — Make processing idempotent: store the delivery/event ID and skip already-seen events (delivery is at-least-once).
 - **Q: Why respond 2xx quickly?** — Providers time out slow endpoints and retry; acknowledge fast, then process asynchronously via a queue.
 - **Q: Is webhook ordering guaranteed?** — No — don't assume events arrive in order; use timestamps/sequence numbers if order matters.
 - **Q: Webhook vs WebSocket?** — Webhook is a one-off server-to-server HTTP POST per event; WebSocket is a persistent bidirectional connection (usually to a browser) for live two-way data.
 - **Q: How do you test webhooks locally?** — Use a tunnel like ngrok to expose localhost, and inspect/replay payloads with webhook.site or the provider's delivery log.
 - **Q: What happens if your endpoint is down?** — The provider retries (usually with backoff) and may eventually disable the webhook or surface failed deliveries to redeliver.
-

 Notes compiled as a quick reference for webhooks.